Agentic LLM Systems:
Reverse and Forward Engineering

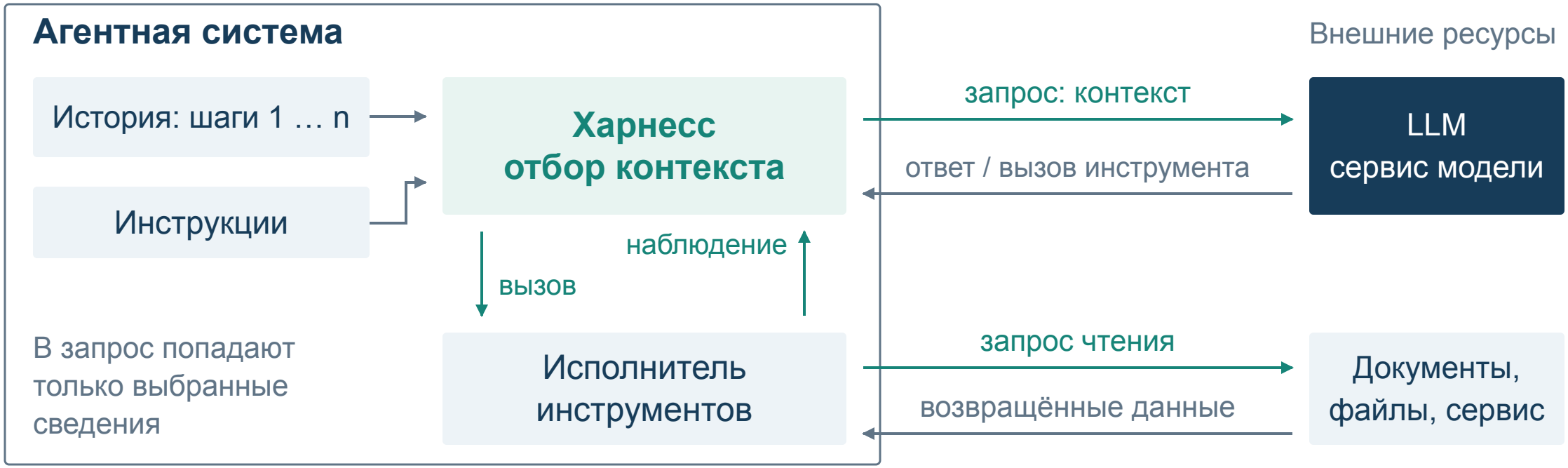
Лекция 4

Контекст-инжиниринг и работа с данными

Алексеев Илья

DeepPavlov

Подготовка следующего входа



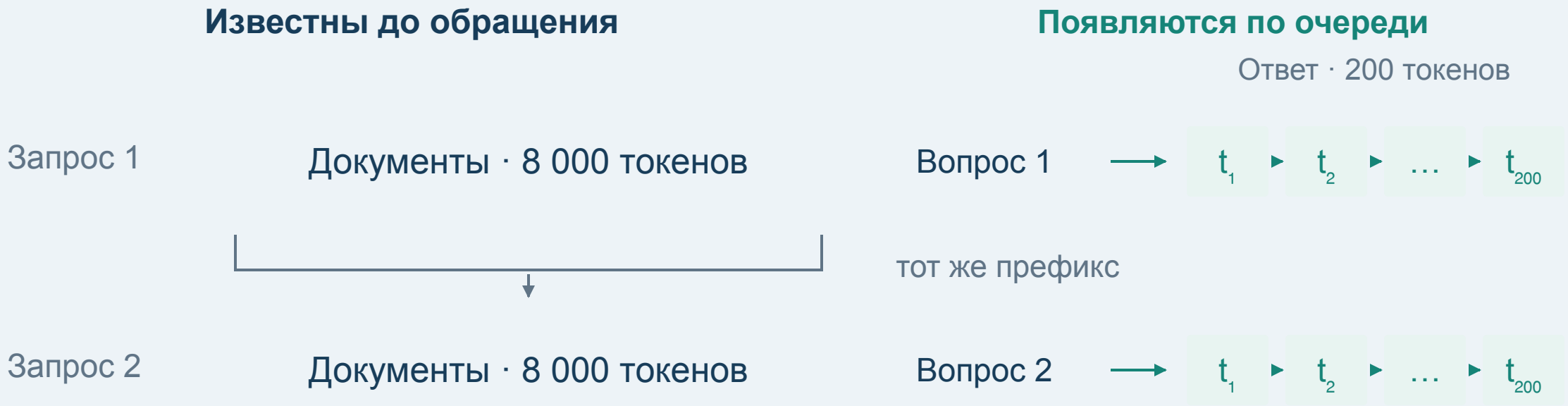
Граница приложения; LLM здесь развёрнута как внешний сервис.

Отбор входа определяет доступную информацию и стоимость обращения.

Контекст и данные LLM-приложения

1. Стоимость контекста: обработка входа и генерация ответа.
2. База знаний в контексте: вместимость и использование сведений.
3. Эмбеддинги и RAG: предварительный отбор документов.
4. Графы знаний: связи между фактами и извлечение подграфа.

Стоимость контекста



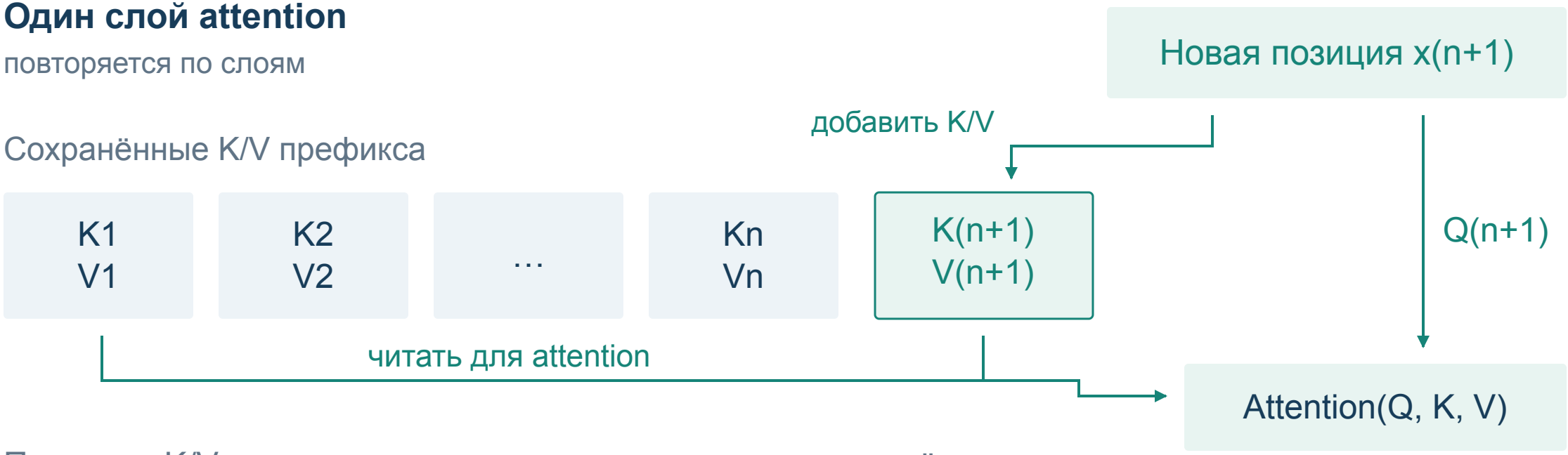
Вход и выход показаны схематично, без масштаба длины или времени.

Вход и выход создают разные составляющие работы.

KV-кэш

Один слой attention

повторяется по слоям



Прошлые K/V сохраняются; их чтение на новом шаге остаётся.

Представления префикса сохраняются; чтение прошлых K/V остаётся.

Prefill и decode

Вход	Вычисления · все слои		Память
Prefill x_1 x_2 x_3 … много известных позиций	**Матричные операции** QKV · attention · FFN → KV-кэш и первый токен	чтение весов, данных ← → запись KV, активаций	Веса KV-кэш
Decode t 1 новая позиция на последовательность	**Все слои на каждом шаге** QKV · attention · FFN → следующий токен	чтение весов и KV ← → запись новых KV и данных	Веса Растущий KV-кэш

Больше операций на загрузку весов: веса переиспользуются

Меньше операций на загрузку весов при малом батче

Сравнение качественное: баланс зависит от батча, размеров и оборудования.

Длинный вход и длинный ответ по-разному используют ресурсы.

Арифметическая интенсивность

I = F / M, единица измерения — FLOP/byte

- Матрица **4096 × 4096**, два байта на вес: **32 MiB**.
- Линейный слой **y = Wx**: 4096 скалярных произведений.
- **F ≈ 2 × 4096² = 33,6 млн FLOP** → около **1 FLOP/byte**.
- Совместная обработка векторов переиспользует загруженные веса.

Умножение и сложение считаем отдельно. M — только однократное чтение весов.

При малом батче decode часто ограничен передачей данных.

Длина контекста и расход памяти

32 слоя · 8 KV-голов · размерность 128 · 2 байта на число

2 × 32 × 8 × 128 × 2 байта = 128 KiB / токен

8192 токена ⟶ 1 GiB K/V

Множитель 2 — ключи и значения. Одна последовательность, без весов и временных буферов.

Длина последовательности влияет и на последующие шаги.

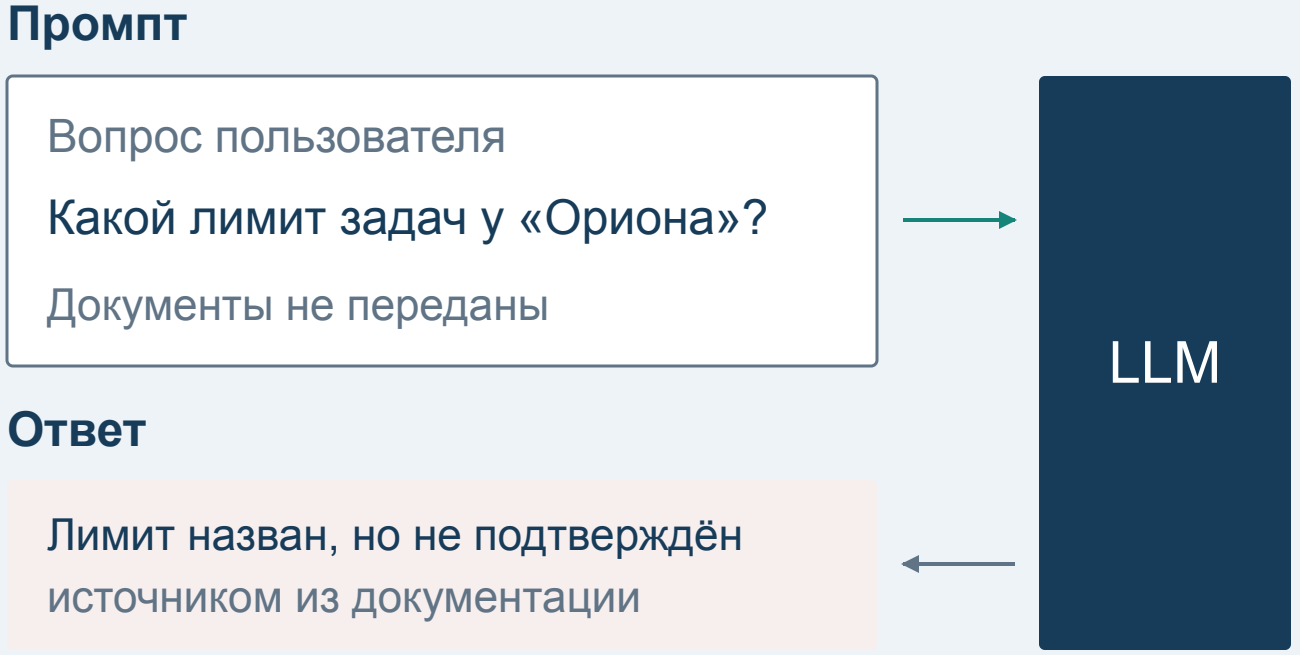
База знаний в контексте

LLM не видит внутреннюю базу

Правдоподобный ответ
может оказаться выдуманным.

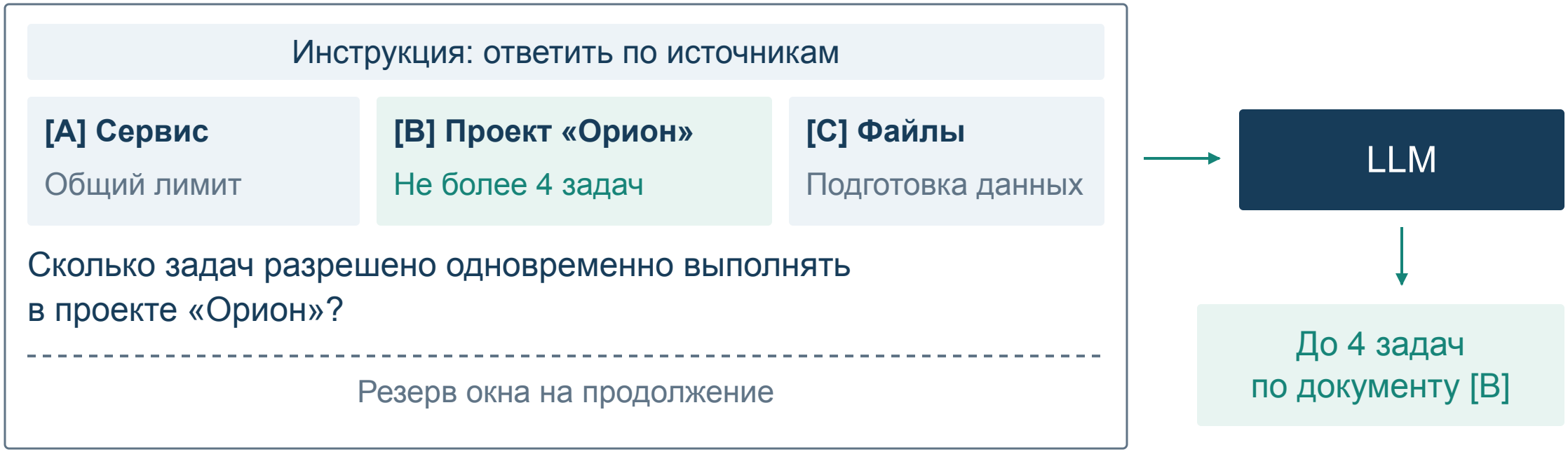
Новые правила проекта
не появляются в весах сами.

Внутренние документы
остались за пределами запроса



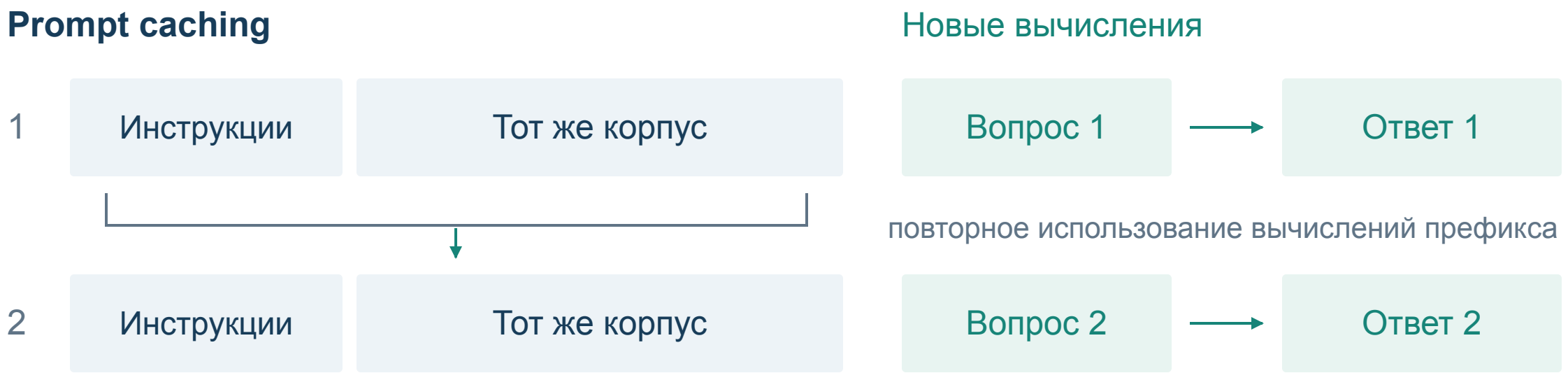
Вопрос сам по себе не даёт модели доступа к внутренним знаниям.

Передаём внутренние знания в запрос



Документ в контексте ещё нужно правильно интерпретировать.

Кэширование статичного префикса



Совместимый префикс → сохранённое вычислительное состояние

CAG · cache-augmented generation (Chan et al.)

Передача корпуса и кэширование вычислений — отдельные решения.

Needle in a haystack

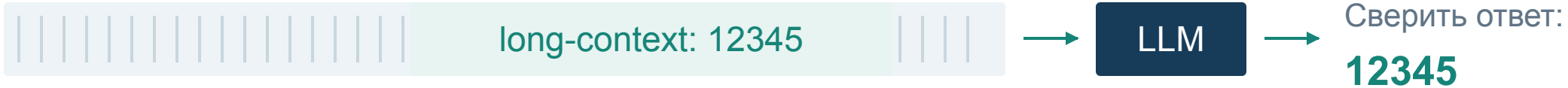
Длинный текст + спрятанный факт + вопрос → LLM → проверка ответа

Kamradt · эссе Paul Graham



Что лучше всего делать в Сан-Франциско?

RULER · пример из статьи



Какое число связано с long-context?

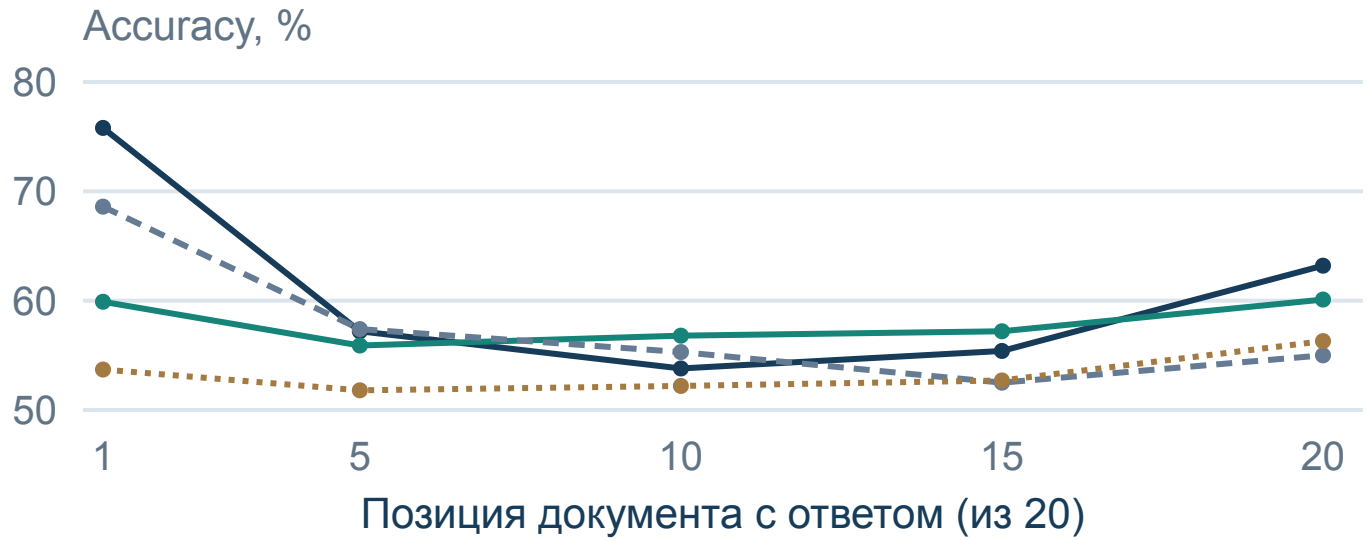
Полосы — фон; вставки — факты. Длина и позиция условны; показаны задания, не оценки.

Тест проверяет извлечение целевого факта в заданных условиях.

Lost in the middle

Lost in the Middle: качество зависит от положения документа

NaturalQuestions-Open · 2 655 вопросов · 20 документов, один с ответом



GPT-3.5-Turbo
Claude-1.3
LongChat-13B (16K)
MPT-30B-Instruct

GPT-3.5: 75,8 → 53,8 → 63,2%
позиции 1 → 10 → 20
провал в середине: −22 п. п.

Liu et al., 2023 · табл. 6 · точки — опубликованные измерения · модели 2023 года

Вместимость окна не гарантирует использования всех сведений.

Эмбеддинги и RAG

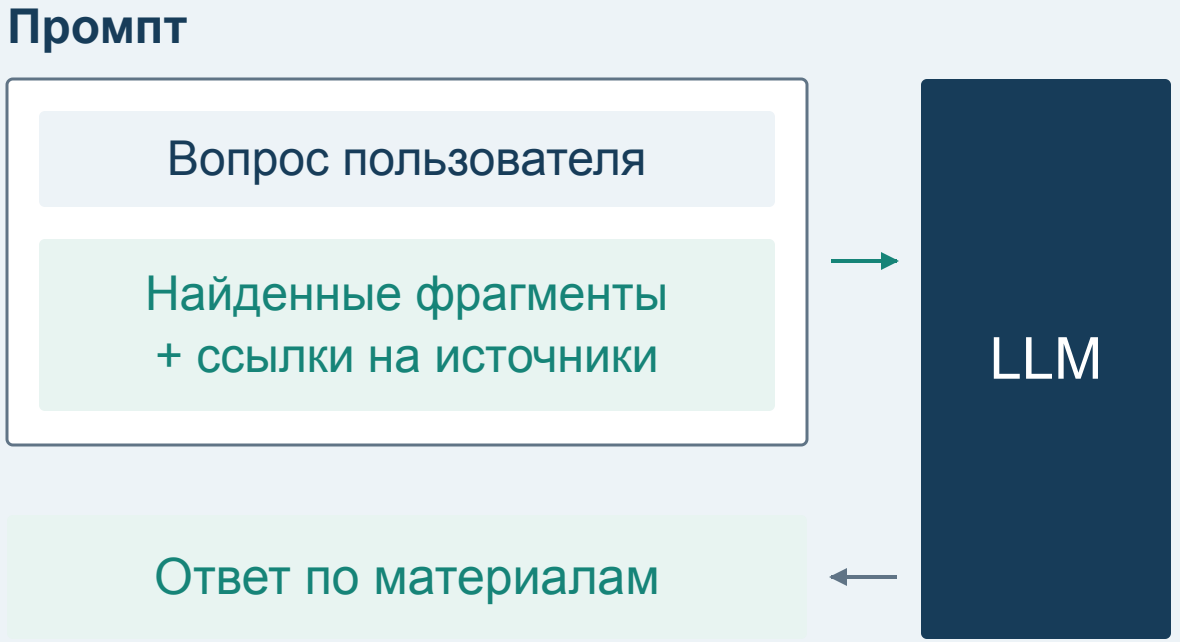
Как дать модели внутренние знания?

Найти материалы под вопрос

Добавить их в промпт

Ответить с опорой на источники

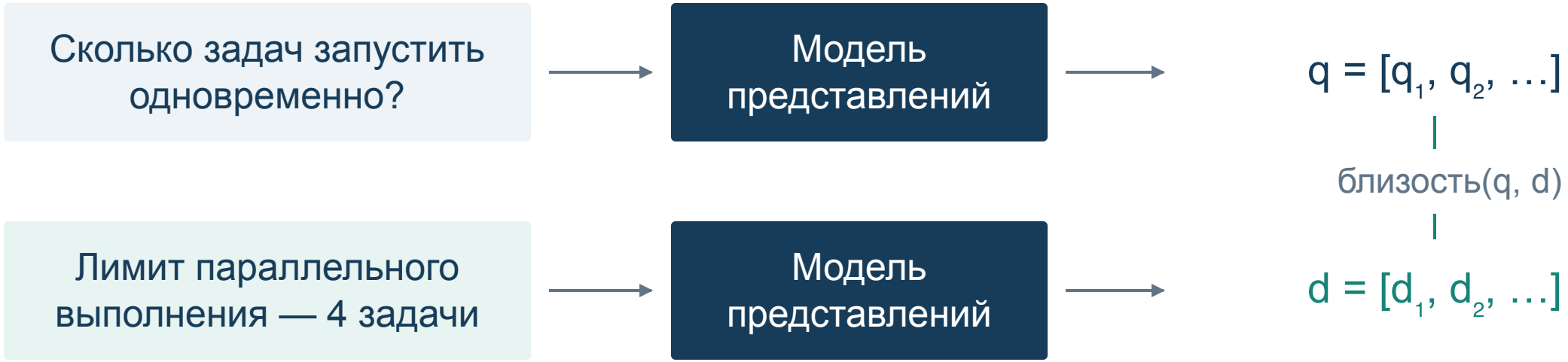
Обновляем документы,
а не переобучаем модель.



RAG добавляет в промпт знания, найденные под текущий вопрос.

Векторизация текста

Тексты

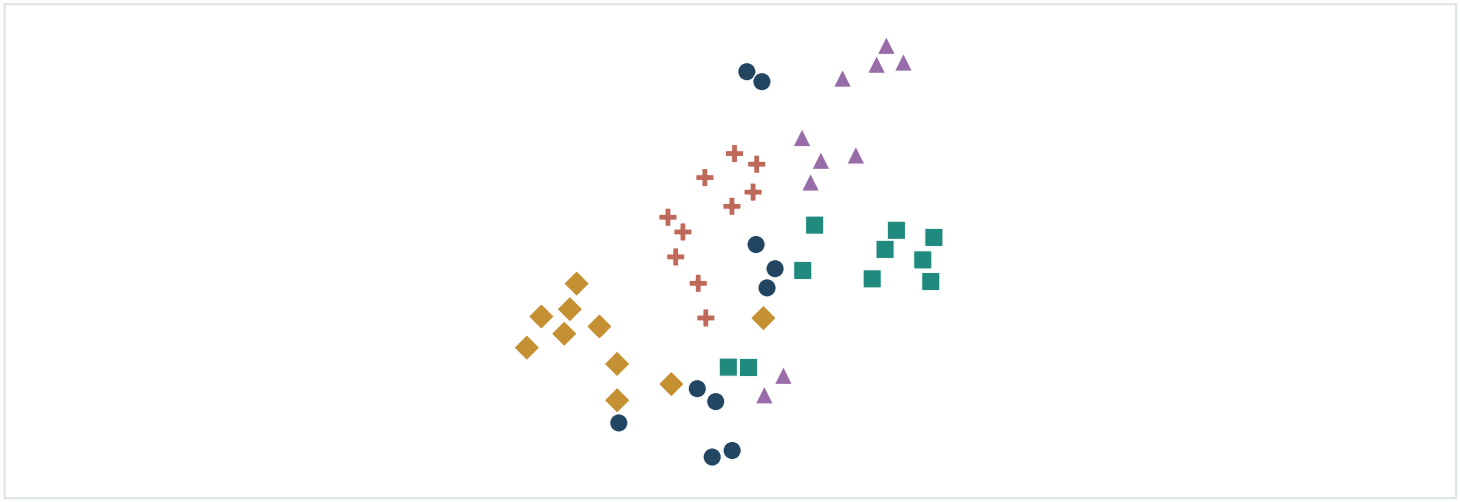


Представления совместимы; качество близости зависит от обучения модели.

Эмбеддинги определяют, какие отношения доступны поиску.

Визуализация эмбеддингов: t-SNE

Учебный синтетический корпус «Ориона»



50 фрагментов · 5 тем

- Задачи и лимиты
- Данные и хранение
- Доступ и роли
- Отчёты и результаты
- Ошибки и диагностика

multilingual-e5-small · 384D → 2D · perplexity = 10 · seed = 42

Поиск — в исходных векторах. Оси и расстояния между группами условны.

Карта помогает исследовать представления; поиск использует исходные векторы.

Подготовка поискового индекса



+ источник и версия

IVF · inverted file

Векторы → списки центроидов

Запрос → ближайшие центроиды

→ поиск в выбранных списках

HNSW

Векторы → граф соседства

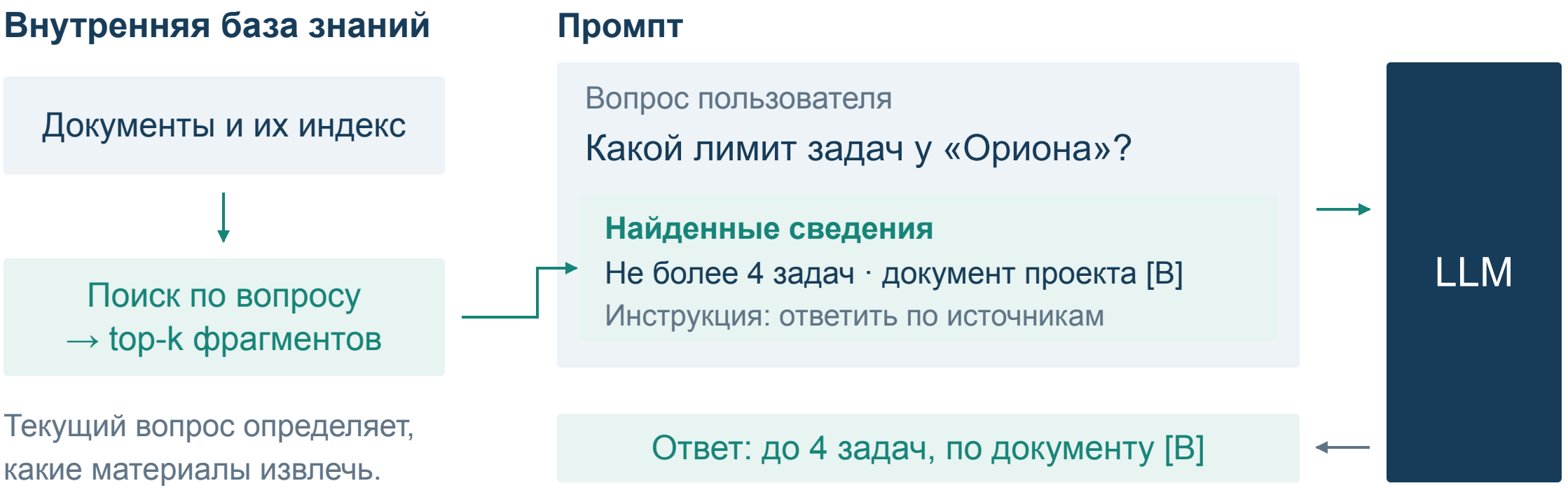
Запрос → обход графа

→ ближайшие кандидаты

Оба подхода ускоряют поиск; ограничение области поиска может снизить полноту.

Индекс задаёт компромисс между скоростью поиска и полнотой кандидатов.

Обработка вопроса в RAG



Найденные фрагменты дают опору для ответа и ссылки для проверки.

Графы знаний

Кто поддерживает очередь «Ориона»?

Условный пример: ответ соединяет сведения из двух документов

Документ A

«Орион» использует **очередь batch**

batch

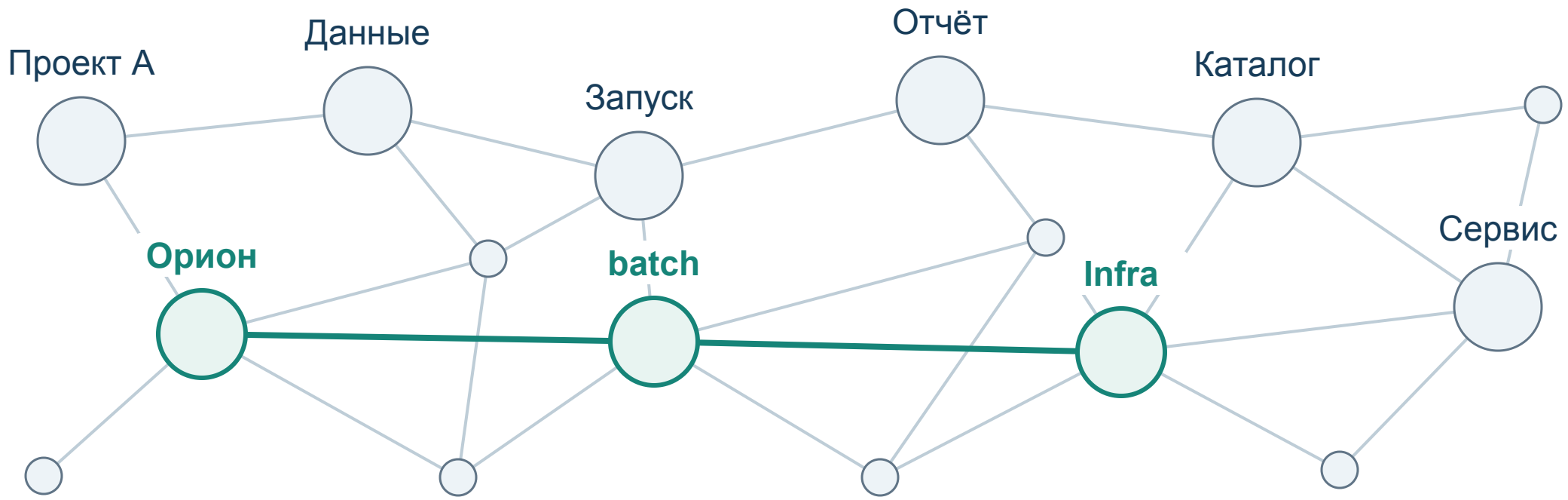
Документ B

Очередь batch поддерживает **команда Infra**

Сущности и их связи можно организовать явно

Граф знаний явно связывает сущности, упомянутые в разных источниках.

Знания как сеть сущностей

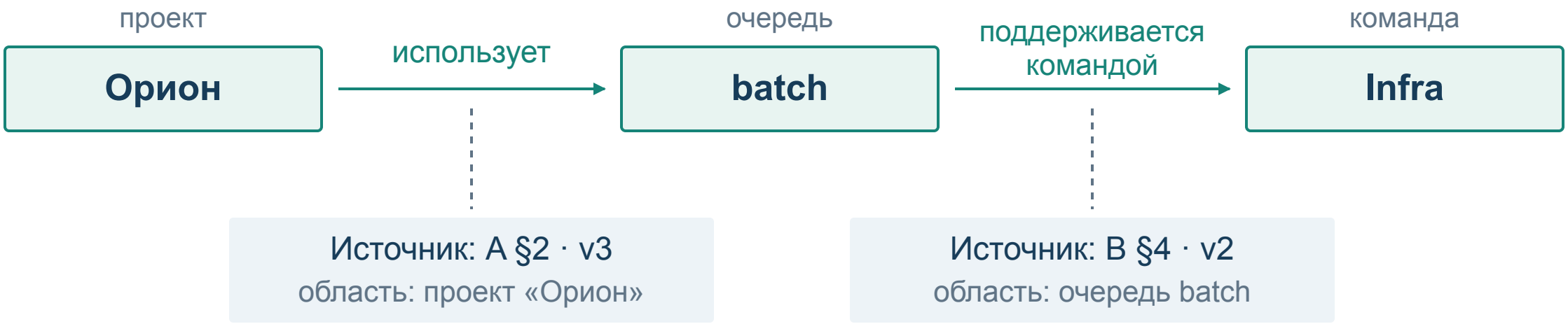


Выделенный путь: Орион → batch → Infra · условный пример

Граф объединяет множество сущностей; вопрос выделяет нужные связи.

Сущности, отношения и источники

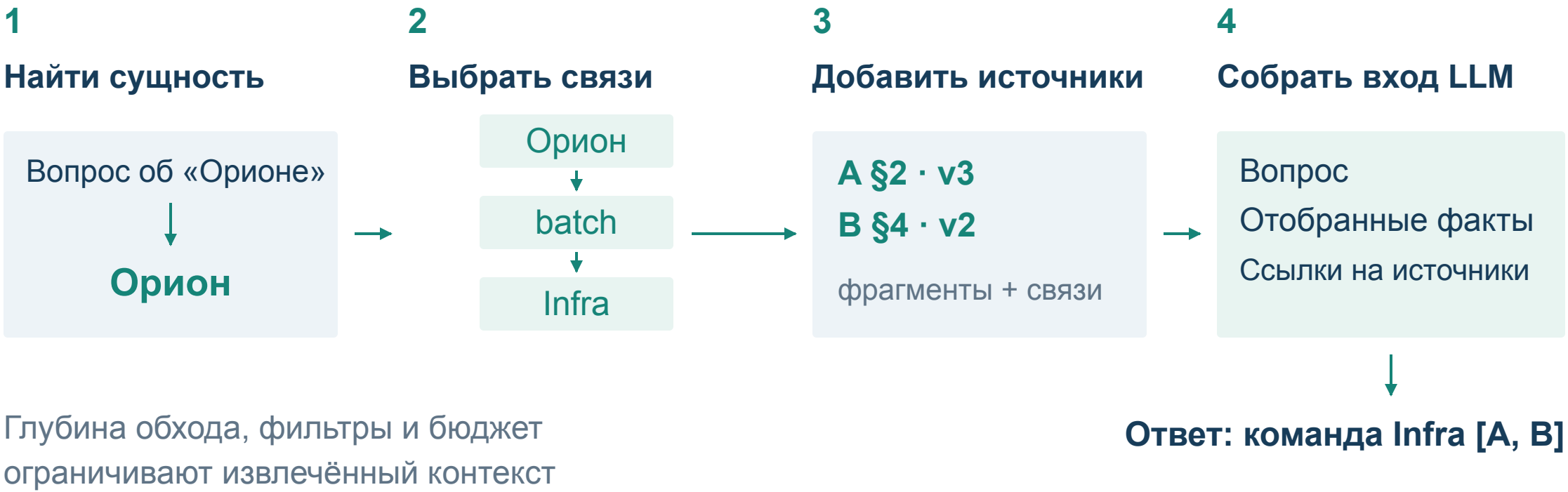
Узлы — сущности; рёбра — предметные отношения



Проверяем извлечение, идентичность сущностей и актуальность связей

Связь должна сохранять смысл, источник и условия применимости.

Извлечение по графу в RAG



Глубина обхода, фильтры и бюджет ограничивают извлечённый контекст

В контекст попадают выбранные связи и подтверждающие фрагменты.

Работа с готовой базой знаний

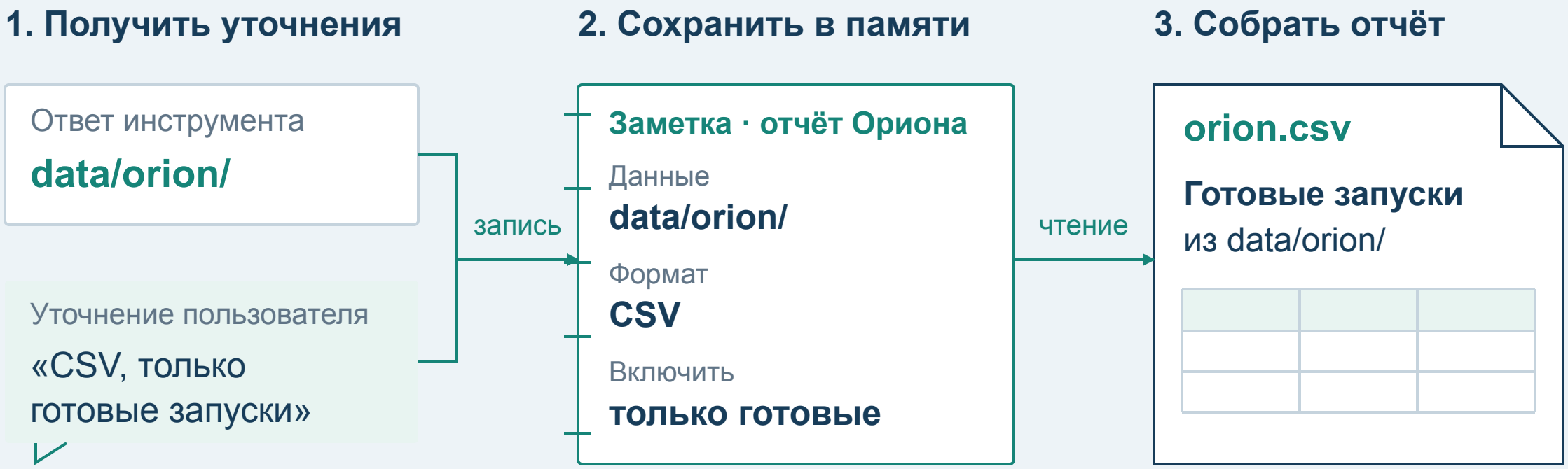
- Корпус в контексте: проверяются вместимость, стоимость и использование сведений.
- RAG: предварительное извлечение определяет доступные свидетельства.
- Граф знаний связывает сущности; retrieval может включать отношения и подтверждающие фрагменты.
- Кэширование повторно использует вычисления для совместимого префикса.

Отбор сведений и повторное использование вычислений решают разные задачи.

Контекст и данные агентного приложения

1. Память: сохранение и извлечение накопленного опыта.
2. Инструкции и инструменты: рост числа способов действия.
3. Discovery: доступ к подробностям по необходимости.
4. Stateful graph: организация исполнения и данных.

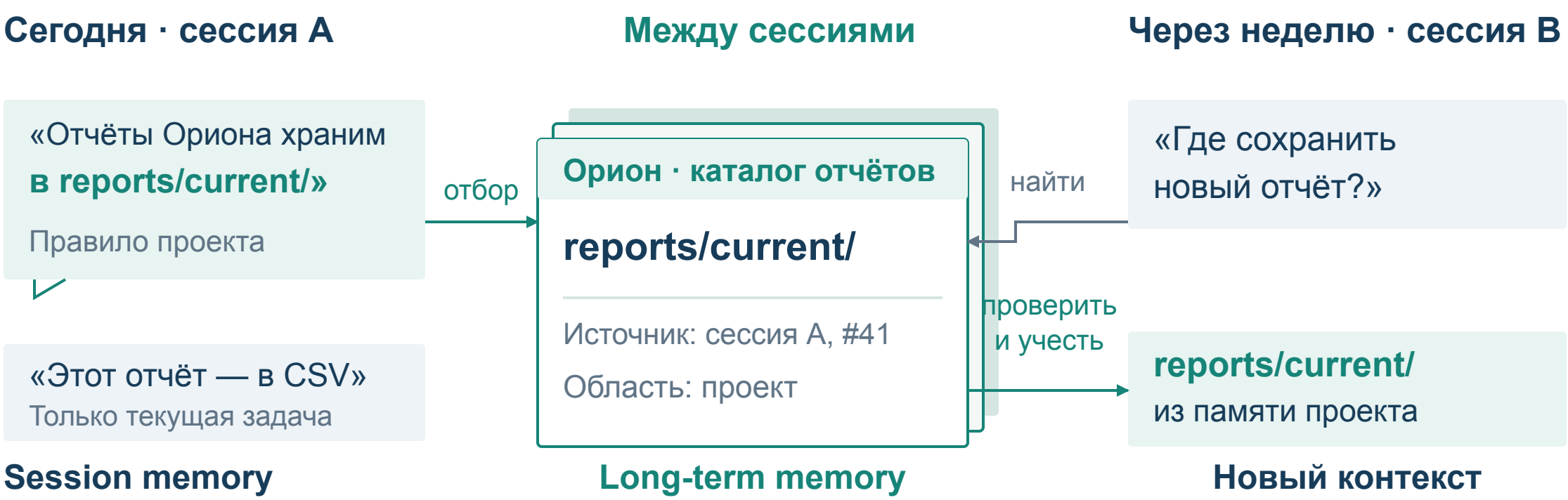
Память агента



Эти сведения относятся к текущему отчёту.

Новые наблюдения требуют сохранения и последующего отбора.

Session memory и long-term memory



Из прошлой сессии в новую задачу переносится выбранное знание.

От наблюдения к воспоминанию

1. Записать наблюдение

Сообщение #42
«Для E7 нужен CSV»
Исходное свидетельство

→

Выделить факт
Формат: CSV · область: E7
Ссылка на #42

→

Сохранить / обновить
Запись: E7 → CSV
Источник: #42

сохранённая запись

2. Использовать позже

Новый вопрос
«Какой формат у E7?»

→

Найти и проверить
Подходит ли запись задаче?

→

Включить в контекст
E7: CSV · источник #42

Сохранение памяти и её включение в запрос — отдельные операции.

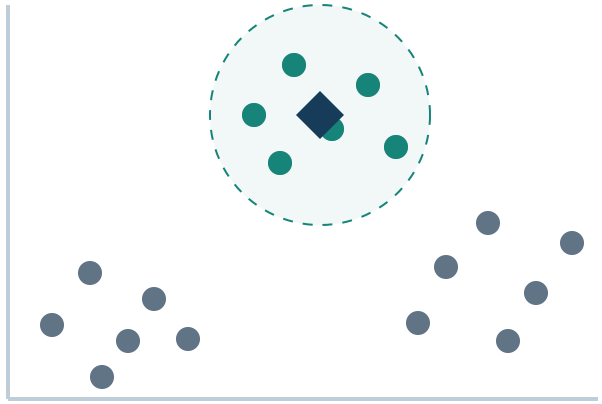
Представления памяти

Сводка



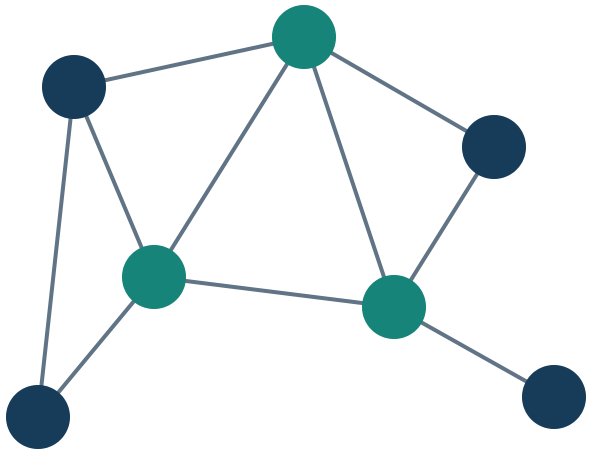
Сжатый текст опыта

Векторный индекс



Поиск похожих записей

Граф



Сущности и отношения

Символические изображения; точки не являются вычисленной проекцией.

Свидетельства, компактное представление и индекс могут сосуществовать.

Гранулярность, дедупликация, инвалидация

Повтор

reports/current/

reports/current/

тот же факт →

Одна запись: reports/current/

Дедупликация

Изменение

Перенос → reports/v2/

→

Сейчас: reports/v2/

История: reports/current/

Прежнее отношение неактуально

Гранулярность: Сообщение | Эпизод | Сессия — условия + источник

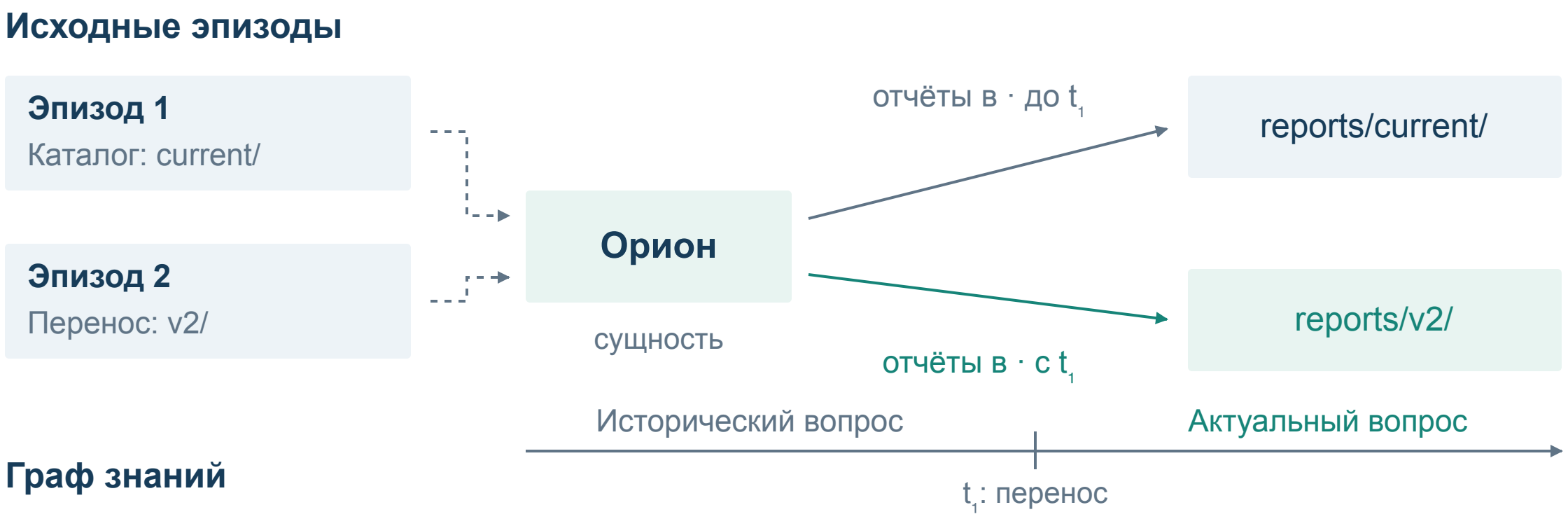
Повтор факта и смена факта требуют разных действий.

mem0: отдельные воспоминания



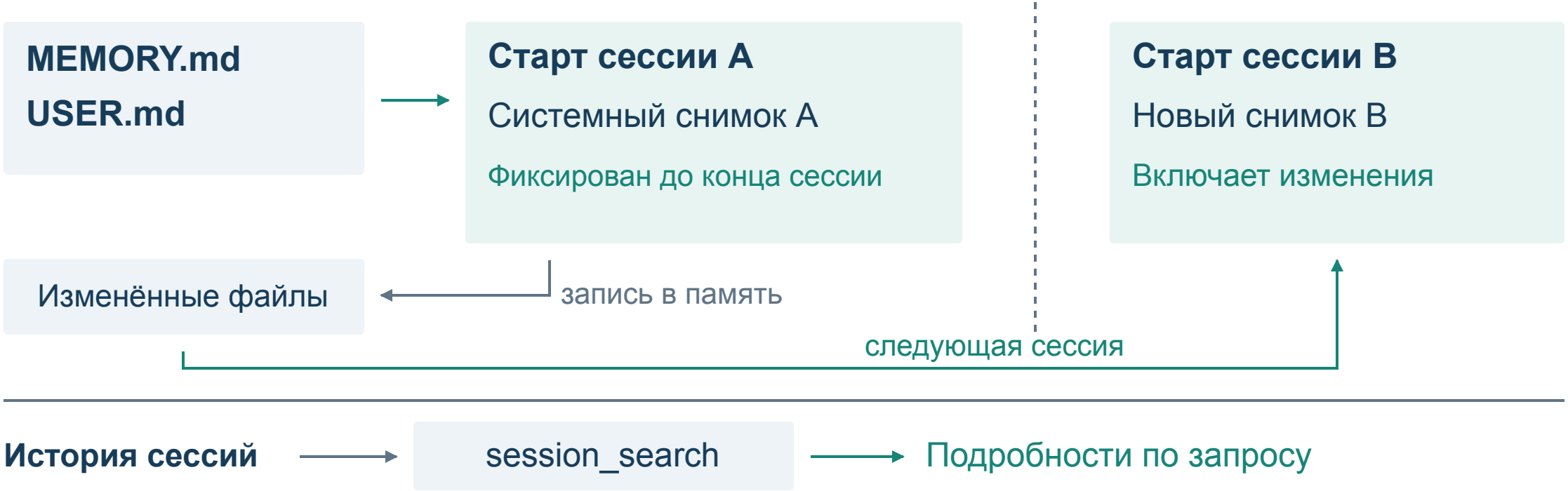
Факт из разговора становится единицей сохранения и извлечения.

Graphiti: факты во времени



Актуальное и историческое состояние факта представлены явно.

Hermes: постоянная часть и поиск истории



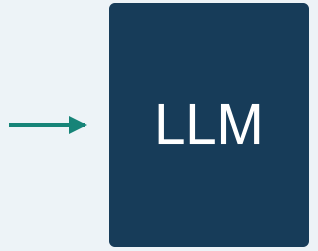
Часть памяти доступна постоянно, подробности извлекаются по запросу.

Инструкции и инструменты

Один запрос к модели · полный каталог

Задача: «Подготовь отчёт по экспериментам Ориона»

Инструкции: как действовать	Инструменты: что вызвать
Отчёт: готовые запуски → CSV	read_runs(project), write_csv(rows)
Файлы: правила редактирования	apply_patch(path, diff)
Сервисы: правила запуска	launch_job(config), get_status(id)



Нужно для отчёта

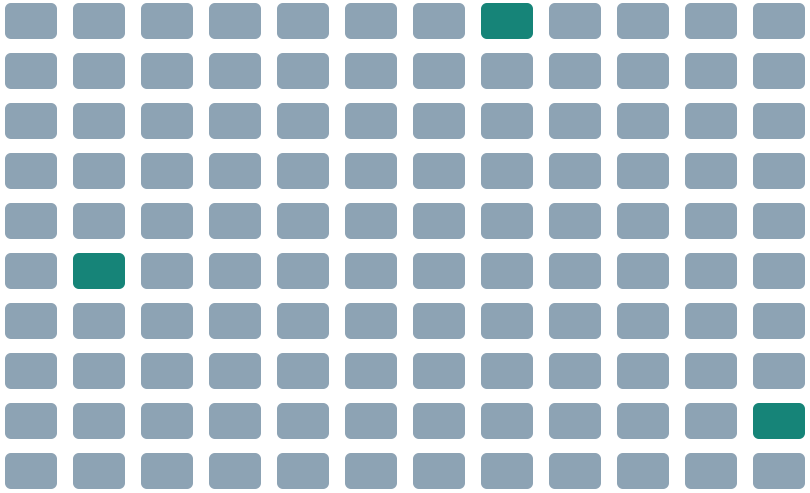
Остальные описания тоже занимают контекст

Инструкции и контракты тоже занимают контекст.

Context bloat: слишком большой каталог

120 инструментов в каталоге

Ячейка = определение инструмента



Условно: 300 токенов на определение

Загружаем весь каталог

120 × 300 = 36 000 токенов

Для этой задачи нужны три

3 × 300 = 900 токенов

Описания занимают контекст ещё до результатов работы инструментов.

Загружая всё заранее, платим за описания, которые сейчас не нужны.

Скилл: пакет инструкций и ресурсов

report-experiments/SKILL.md

```
---
name: report-experiments
description: >
  Подготовка CSV-отчёта
  по экспериментам проекта.
---
# Порядок работы
1. Прочитай references/columns.md.
2. Вызови read_runs(project).
3. Отбери готовые; запиши CSV.
```

Папка скилла · ресурсы необязательны

```
report-experiments/
├── SKILL.md
├── references/
├── scripts/
└── assets/
```

- Организовать инструкции
- Переиспользовать в задачах
- Делиться с командой через Git

description — когда применять; Markdown — как выполнять.

Позднее раскрытие и discovery

Учебный лог · выбор скилла

1 Пользователь

«Собери CSV-отчёт по экспериментам Ориона»

2 Агент

«Загружу скилл report-experiments»

3 Вызов инструмента

read_file("report-experiments/SKILL.md")

Продолжение лога — на следующем слайде.

Уже есть в контексте

name: report-experiments

description:

Подготовка CSV-отчёта
по экспериментам проекта.

Тело SKILL.md ещё не читали.

По description агент выбирает, какой SKILL.md прочитать.

Раскрытие скилла: от инструкции к действиям

Учебный лог · выполнение инструкции

4 SKILL.md	Прочитай references/columns.md. Получи запуски. Оставь готовые и запиши CSV по указанным колонкам.
5 Агент	read_file("report-experiments/references/columns.md") Результат: колонки run_id, score; только status = done.
6 Агент	read_runs(project="orion") → записи запусков Отбор готовых → write_csv(…) → orion.csv

SKILL.md дал порядок действий; справочник уточнил формат результата.

После чтения инструкции агент получает конкретный порядок работы.

Tool Search Tool и deferred loading

Учебный лог · Tool Search доступен с начала

1 Пользователь	«Покажи готовые запуски Ориона»
2 Агент	tool_search_tool_regex({"pattern": "запуски"})
3 Поиск / API	Совпало description: «Прочитать готовые запуски проекта» tool_reference: runs_read → загружен контракт name: runs_read · аргумент project: string
4 Агент	runs_read(project="orion") → данные готовых запусков

Поиск: имена и описания инструментов и аргументов.

defer_loading: true · определение уже в tools API, раскрывается модели после поиска.

Поиск находит определение; после раскрытия агент вызывает инструмент.

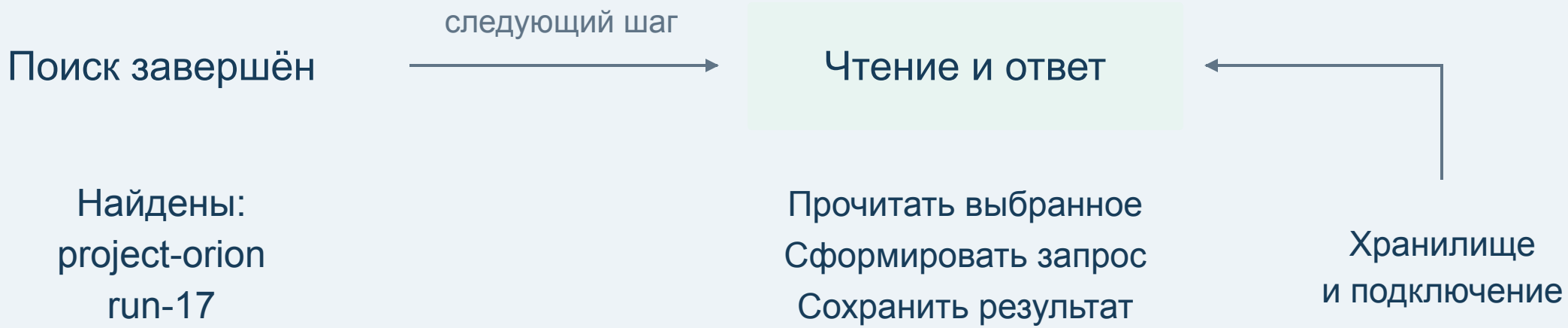
Discovery данных: тот же принцип

Сначала найти нужное, затем прочитать выбранный фрагмент.

- **grep / rg:** найти строки и файлы по текстовому шаблону.
- **SQL:** выбрать нужные строки и столбцы по условиям.
- **Language Server Protocol (LSP):** найти определения символов и места использования через языковой сервер.

Дайте агенту инструменты поиска и точечного чтения данных.

Stateful graph



Данные задачи и доступ к ресурсам нужно передать исполняющему коду.

Управление переходами связано с данными исполнения.

`state`, `deps`, `ctx` и контекст LLM



ctx.state и ctx.deps доступны программе; состав входа модели определяется отдельно.

Доступность данных коду узла не означает их передачи модели.

Контекст исполнения в Pydantic

Pydantic AI — зависимости агента

```
ctx: RunContext[Deps]
ctx.deps
```

Pydantic Graph — состояние и зависимости узла

```
ctx: GraphRunContext[State, Deps]
ctx.state
ctx.deps
```

Контекст исполнения предоставляет объекты программе.

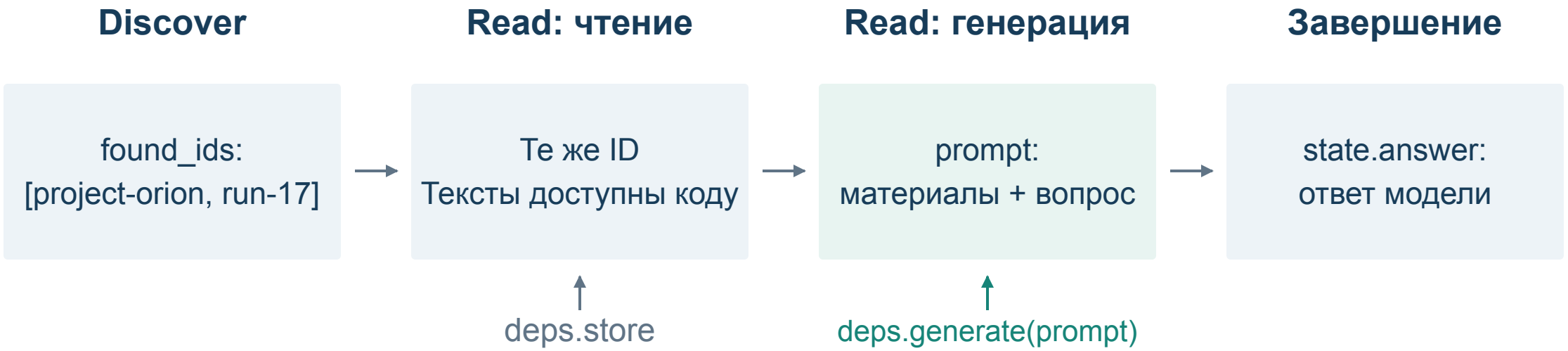
Два узла: обнаружить и прочитать

```
# Discover.run: обнаружить и сохранить ссылки
ctx.state.found_ids = await ctx.deps.store.search(ctx.state.question)
return Read()
```

```
# Read.run: прочитать и явно сформировать вход
if not ctx.state.found_ids:
    ctx.state.answer = "Материалы не найдены."
    return End(ctx.state.answer)
ids = ctx.state.found_ids[:2]
parts = [await ctx.deps.store.read(rid) for rid in ids]
materials = "\n\n".join(f"[{rid}] {t}" for rid, t in zip(ids, parts))
prompt = f"Ответить по материалам.\n{materials}\n{ctx.state.question}"
ctx.state.answer = await ctx.deps.generate(prompt)
return End(ctx.state.answer)
```

Вход модели формируется в явно определённом месте кода.

От ссылки к контексту модели



Ссылки → выбранные тексты → вход LLM → ответ в состоянии.

Состояние приложения и вход модели изменяются по разным правилам.

In-memory, БД, файловая система



In-memory

Объекты процесса
Словарь или индекс

База данных

Записи и устойчивые ID
Клиент или пул соединений

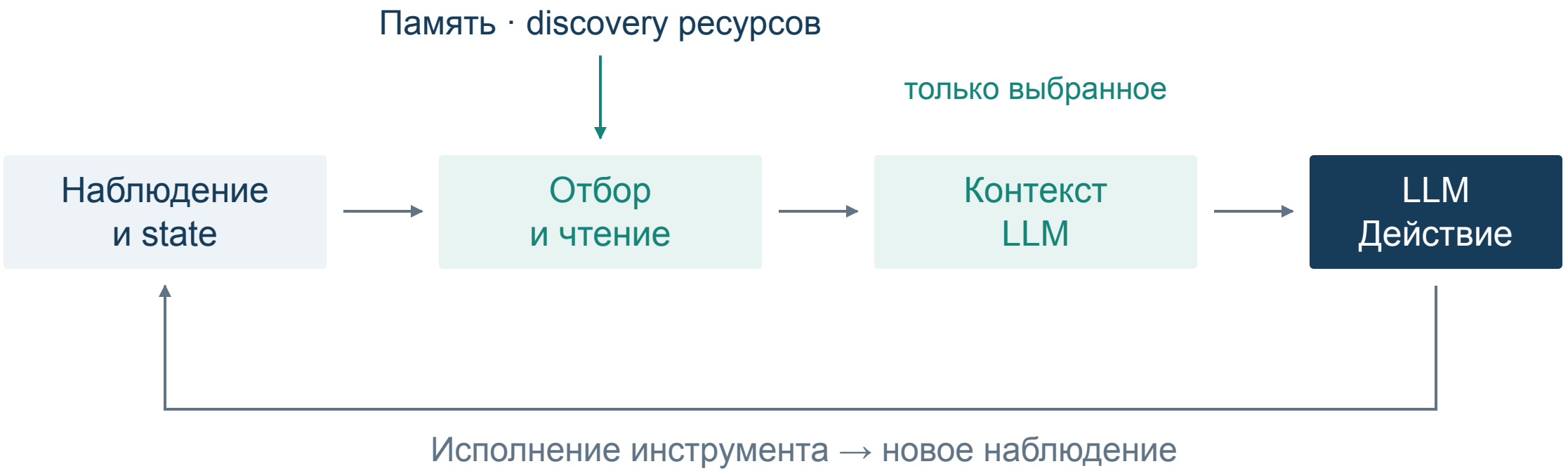
Файлы

Документы и журналы
Пути и метаданные

Состояние выполнения сохраняется отдельно; восстановление графа — дополнительный механизм.

Реализацию хранения можно менять за интерфейсом зависимости.

Контекст на каждом шаге агента



Контекст действия формируется явными операциями приложения.

Управление контекстом и работа с данными

- Объём и состав входа определяют доступные модели свидетельства.
- Кэширование, извлечение, память и позднее раскрытие решают разные задачи.
- Следующий этап — организация последовательности действий при выполнении длинной задачи.

Контекст формируется для обращения; данные живут в приложении.